36118 Applied Natural Language Processing

Master of Data Science and Innovation, UTS

Assessment Task 2B

# End-to-End NLP Project — Technical Report

## Sydney Liveability Explorer

| | |
|---|---|
| **Subject** | 36118 Applied Natural Language Processing |
| **Submission** | AT2 Part B — Group Technical Report |
| **Due Date** | Monday, 4 May 2026, 23:59 |
| **Group** | Group 3, Autumn 2026 |
| **GitHub Repository** | https://github.com/Nelkit/sydney-liveability-ai |
| **Project Link** | https://sydney-liveability-ai.vercel.app/ |

## Team Members


| Last Name | First Name | Student ID | GitHub Username |
|---|---|---|---|
| Chavez Calona | Nelkit | 25609227 | @Nelkit |
| Rodriguez Arciniegas | Juan David | 25418551 | @juandavidrodriguezar |
| Weng | Nian-Ya | | @Amanda1005 |
| Robinson | Luis Gerardo | 25519000 | @LuisRobinsonH |
| Srinivas | Padmasri | | @Psri-01 |
| Liao | Ying-Kai | | @Ying-Kai-Liao |

# Contents

# 1 Executive Summary

*Write this section last. Include one paragraph summarising each of the following: project objectives, data sources, NLP methods, findings, and outcomes.*

---

# 2 Project Objectives and Scope

Sydney Liveability AI addresses a practical and underserved need: helping newcomers to Sydney — international students, migrants, and young professionals — choose where to live based on evidence rather than anecdote. Existing tools such as Domain or Google Maps provide price and location data, but no tool synthesises what residents actually say about their neighbourhoods with objective civic data about safety, transport, and infrastructure in a queryable, plain-language interface.

The project builds a two-layer application. The first layer is an offline NLP pipeline that ingests four verified data sources, runs aspect-level sentiment analysis, generates sentence embeddings, and stores structured results in PostgreSQL and ChromaDB. The second layer is a real-time multi-agent reasoning system that answers natural language questions grounded in that pre-computed evidence, visualised through an interactive suburb-level dashboard.

**In scope for this submission:**

- Aspect-level sentiment analysis over Reddit r/sydney posts (40k+ records) using a transformer-based ABSA model across 8 liveability dimensions.
- Sentence embedding and semantic retrieval via ChromaDB (72,000+ indexed chunks).
- Multi-agent pipeline (CrewAI) with specialised agents for crime, sentiment, GIS, and comparison queries.
- Retrieval-Augmented Generation (RAG) using Claude Sonnet as the synthesis LLM.
- Interactive Next.js dashboard with Leaflet map, per-suburb report views, and an evidence-traced chat interface.

**Out of scope:**

- Automated data refresh pipelines (ingestion is manual; data is a 2024/2026 snapshot).
- User authentication or personalised account storage.
- Suburb coverage beyond those with sufficient Reddit and civic data presence.

---

# 3 Project Phases and Team Contributions

The project was developed over five phases from data ingestion through to evaluation and reporting. Responsibilities were distributed across the team according to each member's technical focus area.

## 3.1 Phase Breakdown Table

| Phase | Tasks | Team Member(s) |
|---|---|---|
| Phase 1: Data | Data acquisition and preprocessing from Reddit (Arc Shift), Community Insights PDF, BOCSAR CSV, TfNSW GTFS, OpenStreetMap (Overpass API), and City of Sydney ArcGIS. | All members |
| Phase 2: NLP Pipeline | Aspect-level sentiment analysis (DeBERTa-v3 ABSA), sentence embeddings (MiniLM), ChromaDB ingestion, spaCy preprocessing, emotion profiling (GoEmotions), coverage detection. | Juan David Rodriguez, Ying-Kai Liao |
| Phase 3: Multi-Agent Pipeline | CrewAI pipeline crew (pre-computation) and query crew (real-time), FastAPI backend, RAG with Open Router API, Synthesiser and Router agents. | All members |
| Phase 4: Frontend | Next.js dashboard, Leaflet map, aspect radar chart, emotion profile visualisation, conversational chat UI, EvidenceDrawer, ReportModal. | Nelkit Chavez, Ying-Kai Liao |
| Phase 5: Evaluation and Report | NLP metrics, RAG quality evaluation (LLM-judge harness), application evaluation, report, poster, peer review. | All members |

### 3.2 Juan David Rodriguez

This team member contributed to the data ingestion and retrieval layers of the pipeline, with a focus on making community-sourced qualitative data accessible to the language model.

The primary contribution was the design and implementation of the full ingestion pipeline for the City of Sydney community consultation PDFs. Starting from structured JSON output produced by the PDF parser, each text chunk was embedded using the `all-MiniLM-L6-v2` sentence transformer model and written to the ChromaDB collection `sydney_liveability` with metadata including suburb, theme, source document, page number, and chunk index. This pipeline processed 1,114 records across three official PDF reports, forming the qualitative backbone of the RAG system.

A secondary contribution was the completion of the Synthesiser agent's ChromaDB retrieval layer, which embeds each user query at runtime, retrieves the top five semantically relevant chunks, and injects them into the LLM prompt alongside structured PostgreSQL outputs and specialist agent results, enabling the model to cite resident quotes and PDF extracts by source and suburb.

### 3.3 Nelkit Chavez

This team member led the backend architecture, civic data pipeline, agent synthesis layer, and the frontend implementation.

Primary contributions included the design and implementation of the Synthesiser agent (`backend/agents/query/synthesiser.py`), which receives structured outputs from all specialist agents and composes a final markdown response using a scenario-aware prompt system distinguishing three query scenarios: `single`, `comparator`, and `out_of_scope`. The Synthesiser iterates all agent outputs to build the cited source list, injects a ChromaDB-retrieved context block before calling the synthesis LLM, and exposes a `SYNTHESIS_DEBUG_MODE` flag for selective output inspection.

Backend contributions also included the GIS agent data layer, the full FastAPI endpoint architecture (`/api/civic`, `/api/chat`, `/api/chat/stream`), and the in-memory caching system with double-checked locking that prevents connection pool exhaustion on Supabase by loading all suburb data once and applying weight changes in pure Python on subsequent requests. A standalone data contribution was the civic ingestion pipeline: facility data from the City of Sydney ArcGIS REST API and walkability scores from the WalkScore API were cleaned, spatially joined with suburb polygons, and ingested into the `suburbs` PostgreSQL table with PostGIS geometry, forming the primary data source consumed by the GIS agent.

On the frontend, this team member designed and built the three-column main layout, the `OnboardingPanel` conversational weight-setting flow, the `MapPanel` with Leaflet polygon rendering and `flyToBounds` behaviour, the `EvidenceDrawer` pipeline trace panel, the `ReportModal` for individual and comparative reports, and the application-wide OKLCH design system with Framer Motion transitions.

### 3.4 Luis Gerardo Robinson Herrera

This team member led the geographic data layer of the pipeline, covering OSM extraction, structured ingestion, and the GIS and Comparator specialist agents.

The primary contribution was the design and implementation of the OSM extraction pipeline (`data_extraction/extract_osm.py`), which queried the Overpass API for ten amenity and leisure POI categories across all 657 Greater Sydney suburbs. The script implements a three-tier resilience strategy comprising multi-endpoint fallback, exponential backoff on rate-limit responses, and per-category retry on timeout. Raw JSON artefacts are persisted to `data/raw/osm/` for auditability. Weighted POI counts were min-max normalised to produce a per-suburb `osm_score` and ingested into the `osm_scores` PostgreSQL table via `backend/scripts/ingest_osm.py` using idempotent upserts.

A second contribution was the GIS specialist agent (`backend/agents/query/gis.py`), which joins three PostgreSQL tables (`suburbs` at 35%, `osm_scores` at 35%, and `transport_scores` at 30%) into a single combined liveability score. The agent includes a keyword parser that filters the OSM payload to a single amenity dimension when the user query targets a specific category. The Comparator agent (`backend/agents/query/comparator.py`) extends this by querying GIS, crime, and sentiment data for two suburbs in parallel and applying a winner-determination rule per category, returning a structured `{suburb_a, suburb_b, comparison, winner}` JSON payload to the Synthesiser.

---

## 4 Data Sources

### 4.1 Primary Data Sources

The pipeline ingests six distinct data sources, each contributing a different dimension of liveability evidence.

**Community Insights PDF**

This source is an official qualitative data source comprising three PDF documents published by the City of Sydney: the Community Insights Report 2024, the Haymarket Vision Engagement Report, and the Haymarket Summary. These reports capture structured resident feedback gathered through formal consultation processes, covering themes such as transport, housing, public spaces,

environment, culture, safety, and social inclusion across suburbs including Haymarket, Sydney, Redfern, Belmore, Ultimo, Burwood, and Camperdown.

Raw PDFs were processed using a custom Python parser (`data_extraction/parse_pdf.py`) that extracted and segmented text into thematic chunks, tagging each record with its source document, suburb, theme, and page number. The pipeline produced four JSON files containing a total of 1,114 records across the three source documents. These structured records were subsequently ingested into ChromaDB as sentence embeddings, enabling the RAG pipeline to retrieve suburb-specific qualitative evidence in response to user queries.

This source provides a dimension of liveability insight that quantitative datasets cannot capture: the lived experience and stated priorities of Sydney residents as documented by their local government. Its official provenance and thematic breadth make it a high-quality complement to the sentiment data collected from Reddit.

**Reddit r/sydney (via PRAW and Arc Shift)**

The Reddit corpus was collected through two complementary extraction paths, both normalised to the same record schema (`text`, `suburb`, `score`, `created_utc`, `url`, `type`). The first path used the PRAW API against the live r/sydney feed, issuing eight aspect-targeted keyword searches per suburb. The second path used the Arc Shift bulk NDJSON archive of historical r/sydney content, partitioned per suburb via string matching against post and comment text. A minimum score threshold of 2 removed low-signal noise without discarding useful short-form resident comments. The pipeline currently covers 565 per-suburb extraction files and 563 matching analysis files under `data/processed/reddit/` and `data/processed/reddit_analyses/`, covering 40,000+ posts and comments.

**BOCSAR Crime Statistics**

Crime data was sourced from the Bureau of Crime Statistics and Research NSW (2024). The dataset was ingested from CSV format (`suburb`, `crime_type`, `year`, `incident_count`, `sa4_area`) via `data_extraction/process_bocsar.py` and loaded into the `bocsar` PostgreSQL table. The Crime agent queries this table to compute per-suburb crime summaries and year-on-year trend indicators (improving/worsening/insufficient data).

**TfNSW GTFS / Transport**

Transport data was sourced from TfNSW GTFS feeds combined with OSM transit data (2026). The ingestion pipeline (`data_extraction/extract_transport.py`) populates the `transport_scores` table with bus stops, train stations, light rail stops, bike path kilometres, average commute minutes, average services per hour, and a normalised transport score per suburb.

**OpenStreetMap (OSM via Overpass API)**

Amenity counts were extracted from OpenStreetMap using the Overpass API and OSMNX (2026). Amenity categories include cafe, restaurant, gym, school, hospital, pharmacy, library, park, playground, and sports centre. These are stored in the `osm_scores` table and contribute to the GIS agent's composite infrastructure score:

```
combined_score = (facilities_score * 0.35) + (osm_score * 0.35) + (transport_score * 0.30)
```

**City of Sydney ArcGIS**

City of Sydney facility data was retrieved from the ArcGIS REST API (2026), including libraries, car-share bays, mobility parking, sports facilities, and walkability scores per suburb. These are stored in the `suburbs` table with PostGIS geometry (MultiPolygon, EPSG:4326), enabling the map layer and CBD proximity scoring via `ST_Distance`.

### 4.2 Data Limitations and Ethical Considerations

**Community Insights PDF**

This source presents limitations that should be considered when interpreting pipeline outputs. Text was extracted from structured reports rather than raw survey responses, meaning some granularity is lost in the aggregation process. The reports summarise resident feedback through an editorial lens applied by the City of Sydney, which may omit minority opinions or nuanced dissenting views. Geographic coverage is uneven: the majority of records are concentrated in Haymarket (383 records) and Sydney CBD (108 records), while suburbs such as Camperdown and Burwood contribute only a handful of entries. Suburb attribution relied on named entity recognition and heuristic matching during PDF parsing, which introduces a small margin of error in cases where suburb names are ambiguous.

From an ethical standpoint, all three PDFs are publicly published by the City of Sydney and were used solely for research purposes. The community feedback they contain was gathered through formal, consented consultation processes.

**Reddit**

Reddit r/sydney is a large but self-selected community. Posts reflect the perspectives of active users who choose to write publicly, which may skew toward residents with strong opinions (positive or negative). There is geographic bias toward inner-city and well-known suburbs: popular areas such as Newtown, Glebe, and Surry Hills have hundreds of indexed chunks, while peripheral suburbs may have few or none. When Reddit coverage for a dimension falls below the coverage threshold, the system routes to a structured fallback rather than fabricating a sentiment score.

From a privacy standpoint, all Reddit content used was publicly posted and accessed through the official API or a publicly distributed bulk archive. No private messages or removed content were included.

**BOCSAR and Civic Data**

Crime and infrastructure datasets are snapshots from 2024–2026. They do not reflect changes that may have occurred after the ingestion date. The BOCSAR data uses SA4-level geographic groupings for some crime categories, which may not perfectly align with suburb-level granularity.

---

## 5 NLP Methods and Techniques

### 5.1 Text Preprocessing

All text data was preprocessed using spaCy before entering the NLP pipeline. The preprocessing steps include tokenisation, lemmatisation, and named entity recognition (NER) to extract and normalise suburb name references. This NER step is critical because suburb mentions in Reddit posts are often informal (e.g., "the Gong", "inner west") and must be mapped to canonical suburb names for downstream filtering and retrieval.

For RAG ingestion, text was chunked using LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 200 characters and 20-character overlap. This strategy balances retrieval granularity (small enough to isolate a specific claim) with context preservation (overlap avoids hard cuts mid-sentence). Chunk IDs are deterministic on `(source, post_id, chunk_index)`, enabling idempotent re-ingestion without duplicating records in ChromaDB.

## 5.2 Considered Approaches: Traditional NLP

Three traditional NLP approaches were evaluated during the design phase and ultimately not adopted as primary implementations.

**VADER Sentiment Analysis** (Hutto & Gilbert, 2014) is a lexicon-based rule system that assigns positive, negative, and neutral polarity to text. While fast and well-established, VADER produces a single document-level score and cannot attribute sentiment to a specific aspect of a text. A Reddit post saying "the food is great but rent is brutal" would receive a near-neutral VADER score, losing the critical distinction between high food-and-cafe sentiment and low affordability sentiment. This limitation makes VADER insufficient for a system requiring 8-dimension aspect-level scores per suburb.

**LDA Topic Modelling** (Latent Dirichlet Allocation, Blei et al., 2003) was considered for discovering thematic structure in the Reddit corpus. LDA is an unsupervised method that identifies co-occurring word distributions as latent topics. However, LDA performs poorly on short, noisy, and informal Reddit text, which lacks the document length required for stable topic inference. Topic labels also require manual interpretation and produce inconsistent results across different random seeds, limiting reproducibility and evaluation rigour.

**TF-IDF Keyword Extraction** was considered as a lightweight alternative to semantic search. TF-IDF measures term frequency weighted by inverse document frequency, highlighting distinctive terms per suburb. While useful for exploratory analysis, TF-IDF has no semantic understanding: it cannot recognise that "flat", "apartment", and "unit" refer to the same concept, or that "dodgy" and "unsafe" are semantically related to the safety dimension. This limits its utility for retrieval in a system where users ask open-ended natural language questions.

## 5.3 Implemented Approaches: Modern NLP

### 5.3.1 Reddit Data Collection via Arc Shift

Two complementary extraction paths feed a single per-suburb JSON shape under `data/processed/reddit/`. The first path uses PRAW against the live r/sydney API, issuing eight aspect-targeted keyword searches per suburb (safety, food and cafe, nightlife, affordability, transport, community, noise, green space). PRAW is rate limited and only reaches the most recent few hundred posts per query, so the second path uses the Arc Shift bulk dump, an NDJSON archive of historical r/sydney content. We partition Arc Shift records per suburb by string matching the suburb name and known variants against post and comment text. Both paths normalise to the same record shape (`text`, `suburb`, `score`, `created_utc`, `url`, `type`) so downstream code only handles one schema.

The pipeline currently covers 565 per-suburb extraction files in `data/processed/reddit/` and 563 matching analysis files in `data/processed/reddit_analyses/`. A floor of `score >= 2` removes the bulk of low-signal noise without dropping useful one-line residents' comments.

### 5.3.2 Aspect-Level Sentiment Analysis (DistilRoBERTa)

The per-suburb analysis (`backend/core/nlp/pipeline.py`) runs four transformer-backed stages followed by a deterministic cross-modal fallback. Although the section heading retains the DistilRoBERTa label used in early planning, the implemented sentiment model is `yangheng/deberta-v3-base-absa-v1.1`, chosen because it scores polarity jointly with the aspect span instead of treating the whole text as a single signal.

**Aspect classification.** `backend/core/nlp/aspects.py` runs `facebook/bart-large-mnli` zero-shot classification over each chunk against eight liveability dimensions: safety, food and cafe, nightlife, affordability, transport, community, noise, and green space. A confidence threshold of 0.30 keeps the top label; below that the chunk is tagged "general" and excluded from per-aspect aggregation. The same aspect catalogue drives ingestion tagging, search-time filters, and the chat agent's dimension routing, so changing it propagates consistently.

**Aspect-Based Sentiment Analysis.** `backend/core/nlp/sentiment.py` replaces the previously planned whole-text VADER scoring with DeBERTa-v3 ABSA. ABSA scores each (text, aspect) pair separately, so a single Reddit post saying "the food is great but rent is brutal" registers positive food-and-cafe sentiment and negative affordability sentiment instead of a neutral average. Polarity maps to a 0–1 scale where 0 is negative, 0.5 is neutral, and 1 is positive. Two routing rules guard against ABSA's known weaknesses on short, noisy text:

- A confidence floor of 0.55 on the ABSA softmax. The model is overconfident on terse Reddit one-liners, and below this floor the (text, aspect) pair is routed to a BART-MNLI fallback path.
- A word-count floor of 10. Texts shorter than this skip ABSA entirely and use the fallback path.

The fallback contribution is also dampened in the per-suburb aggregate so a corpus of short posts cannot drown out the longer ABSA-confident signal.

**Coverage detection.** `backend/core/nlp/coverage.py` answers a different question from classification: not *how is the topic discussed?* but *is the topic discussed at all?* Each dimension has a hand-written prototype sentence. Chunk embeddings and prototypes are compared by cosine similarity using MiniLM; the mean of the top-10 similarities per dimension determines the coverage tier per the thresholds in Table 2. Tier thresholds were tuned against a 30-suburb hand-labelled sample. This separation matters because zero ABSA pairs can mean two things: nobody talks about the topic, or the BART-MNLI top label landed on a different aspect. Coverage distinguishes the two cases honestly.

**Table 2: Coverage tier thresholds (mean of top-10 cosine similarities to the dimension prototype).**

| Tier | Threshold | Source field | Meaning |
|---|---|---|---|
| strong | mean >= 0.40 | reddit | enough Reddit signal to score the dimension directly |
| weak | 0.25 <= mean < 0.40 | reddit or fallback | some chatter; score with reduced confidence |
| none | mean < 0.25 | none or fallback | no Reddit signal; route through cross-modal fallback or null |

**Emotion profile.** `backend/core/nlp/emotions.py` runs DistilBERT fine-tuned on GoEmotions to attach a 27-label emotion distribution per chunk (anger, joy, sadness, fear, gratitude, etc.). The chunk-level distributions are aggregated to a per-suburb softmax that the chat agent uses for narrative hooks. GoEmotions is lightweight relative to ABSA, so it runs unconditionally on every chunk.

**Cross-modal fallback.** `backend/core/nlp/fallback.py` defines a static per-dimension dispatch table for when Reddit is silent on a dimension. Safety routes to BOCSAR crime-statistic percentile anchors; transport and walkability route to OSM scores; demographics-leaning dimensions route

to City of Sydney ArcGIS suburb rows. When neither Reddit nor a fallback produces a signal, the dimension's score is null and source is "none", and the chat agent treats the pair as missing data rather than fabricating a value.

**Output schema.** The pipeline writes one JSON file per suburb under `data/processed/reddit_analyses/`. Each `SuburbAnalysis` carries the suburb name, post count, fetch timestamp, an `aspects` dict keyed by dimension (with score, mentions, confidence, coverage, and source per entry), an emotions softmax over 27 GoEmotions labels, an LLM-generated narrative summary, and a list of curated quote sources.

### 5.3.3 Sentence Embeddings (all-MiniLM-L6-v2) and ChromaDB

`backend/db/chromadb.py` manages the `sydney_liveability` collection, currently around 72,000 chunks built by two ingestion scripts. Embeddings come from `sentence-transformers/all-MiniLM-L6-v2`, accessed via a process-wide singleton in `backend/core/embeddings.py` so the coverage detector and the ChromaDB writer load the model exactly once (saving roughly 500 MB of repeated initialisation). The MiniLM choice trades a small amount of retrieval quality for a 5x smaller index and faster ingestion than larger encoders, which matters when the index rebuilds take approximately 10 hours from scratch.

Reddit ingestion (`backend/scripts/ingest_reddit.py`) reads the per-suburb JSON arrays, splits each post with a 200-token chunker (20-token overlap), classifies each chunk's top liveability dimension via BART-MNLI (threshold 0.30, else "general"), and upserts into ChromaDB with `source="reddit"`. Chunk IDs are deterministic on `(source, post_id, chunk_index)` so re-running the script replaces rows in place rather than duplicating them.

Sentiment ingestion (`backend/scripts/ingest_sentiment.py`) ingests the per-suburb narrative summaries plus the curated quote chunks from `SuburbAnalysis.sources`, tagged `source="sentiment"`. The chat agent prefers these when looking for clean, attribution-friendly citations, falling back to the bulk Reddit chunks when none match the question.

Each chunk carries metadata for suburb, dimension, source, url, and score, so all retrieval filters run server-side in ChromaDB rather than as a Python post-filter on a wide query result.

### 5.3.4 Multi-Agent Pipeline (CrewAI)

The system uses two CrewAI crews with distinct responsibilities. The **pipeline crew** runs the per-suburb NLP analysis described above and writes `SuburbAnalysis` files; this is offline and runs whenever a suburb's Reddit corpus is refreshed. The **query crew** (`backend/crews/query_crew.py`) responds to chat turns in real time and is the focus here.

Architecturally the query crew follows a two-layer agentic RAG pattern in the sense of Du et al. (2026): an upper agent layer (router, specialists, synthesiser) decides what to retrieve, and a lower tool layer exposes retrieval interfaces the agents can call adaptively. The query crew is sequential: a router, three specialists, an optional comparator, and a synthesiser. The architecture reflects best practices for building robust agentic systems by separating concerns into specialized roles, as proposed by **S. and Zhang (2024)**, ensuring that the LLM has a well-documented and easy-to-use interface through modular tools

The **Router** is entirely deterministic ( regex and keyword-based with no LLM call ) keeping routing latency near zero and reducing API costs. Suburb detection uses a case-insensitive regex over the full suburb list, sorted by length (longest-first) to prevent substring conflicts. The router classifies

questions into one or more of: `crime`, `sentiment`, `gis`, `comparator`, or `out_of_scope`, and extracts all suburb mentions.

The **Crime agent** queries the `bocsar` PostgreSQL table to compute per-suburb crime summaries by category and year-on-year trend indicators (improving/worsening/insufficient data).

The **Sentiment agent** is the agentic retrieval entry point — a CrewAI ReAct agent with three tools: `search_posts(suburb, dimension, query, k)` for dense semantic ChromaDB search; `get_suburb_aspect(suburb, dimension)` for structured aspect lookups; and `compare_suburbs(suburbs , dimension)` for ranking an input list by score. The agent decides which tools to call and in what order based on the question, satisfying the three principles of agentic autonomy identified by Du et al. (2026): autonomous strategy, iterative execution, and interleaved tool use. A prompt-level grounding rule nudges the agent to call `search_posts` at least once on qualitative questions, but the agent retains full decision-making authority on ambiguous cases.

The **GIS agent** queries `suburbs`, `osm_scores`, and `transport_scores` to compute the composite infrastructure score and returns detailed facilities, amenity counts, and transport data.

The **Comparator agent** produces side-by-side rankings when two or more suburbs are mentioned, with dimension-specific winner logic (lower crime, higher sentiment, higher GIS score).

The **Synthesiser** composes the final markdown response using scenario-aware prompts for three cases: `single` (one suburb), `comparator` (two or more suburbs), and `out_of_scope`. It iterates all agent outputs to build the source list, injects ChromaDB chunks retrieved by suburb as additional context, and generates a response that is concise, markdown-formatted, and explicitly cites its evidence sources.

#### 5.3.5 Retrieval-Augmented Generation (RAG + LLM)

The synthesiser composes the final answer from the structured specialist outputs returned by the upper agent layer. For sentiment turns this output is the JSON the ReAct agent emits when it stops: a per-suburb payload of the dimensions the agent actually queried, an emotion softmax attached server-side, the suburb name, an overall sentiment label, a list of grounded quotes drawn from `search_posts` results, and an `evidence_trace`.

The trace is recorded as a side channel rather than as a contract: each tool wrapper times its call and appends a `TraceEntry` (`step`, `tool`, `arguments`, `result_count`, `result_preview`, `elapsed_ms`) to a per-call `ContextVar` that `_query_sentiment_impl` sets before `crew.kickoff` and drains afterwards. The trace mirrors what the agent actually did — it does not pre-decide the loop. The synthesiser sees both the agent's settled conclusions (the dimensions, the quotes) and the trace of how it got there, and the existing citation rule ("cite at least one trace entry per named-suburb claim") applies naturally without constraining the agent's decision-making.

The chat LLM is configurable via environment variables. During development the system routes through OpenRouter, which provides access to free-tier models used for testing and iteration. For production, the system is configured to use paid OpenRouter models or OpenAI models, leveraging API credits provided by UTS for this subject. The synthesiser's prompt includes the question, a database-derived context block (facilities, transport, OSM amenity counts), and the structured specialist outputs.

A spatial-intent gate keeps the synthesiser from responding authoritatively to out-of-scope questions: when `agent_outputs["router"]["categories"]` equals `["out_of_scope"]`, the prompt drops the spatial-context block entirely and the answer reverts to a polite refusal listing example valid ques-

tions.

### 5.4 Justification: Why Modern Over Traditional

Present an academic argument for each design choice: DistilRoBERTa over VADER, sentence embeddings over TF-IDF, RAG over keyword search, and CrewAI for modular orchestration. Cite Lewis et al. (2020), Reimers & Gurevych (2019), and Hutto & Gilbert (2014).

---

## 6 Findings and Evaluation

### 6.1 NLP Pipeline Results

The offline NLP pipeline processed Reddit data for over 560 Sydney suburbs, producing aspect-level sentiment scores across 8 dimensions (safety, food and cafe, nightlife, affordability, transport, community, noise, green space). Coverage is strongest for inner-city suburbs with high r/sydney engagement. For well-covered suburbs such as Newtown, Glebe, and Surry Hills, all 8 dimensions return ABSA-confident scores with `strong` coverage tiers. For peripheral suburbs, dimensions with insufficient Reddit signal are routed to the cross-modal fallback (BOCSAR for safety, OSM for transport) or returned as null with `source="none"`, preventing fabricated scores.

The emotion profiling pipeline (GoEmotions, 27-label) produces per-suburb softmax distributions that reveal affective patterns beyond simple positive/negative polarity. High-density residential suburbs tend to exhibit elevated `joy` and `gratitude` scores correlated with strong food and cafe and community dimensions, while suburbs with higher crime rates show elevated `fear` and `anger` in the emotion profile — consistent with concurrent BOCSAR data. These correlations provide qualitative validation that the sentiment pipeline is capturing real signal rather than noise.

The composite liveability scoring formula combines all six dimensions with user-defined weights:

```
liveability = (safety * w_safety) + (transport * w_transport) + (lifestyle * w_lifestyle)
            + (affordability * w_affordability) + (nightlife * w_nightlife) + (proximity *
    w_proximity)
```

All component scores are normalised to [0.0, 1.0]. The safety score inverts crime counts (`1 - crime/max_crime`); proximity uses PostGIS `ST_Distance` from the suburb centroid to the CBD (`lat =-33.8688, lng=151.2093`), clamped at 35 km. The in-memory cache (`_RAW_CACHE` with double-checked locking) ensures that weight changes trigger instant re-ranking in pure Python without additional database calls.

### 6.2 NLP Model Performance Metrics

The evaluation harness (`backend/scripts/run_agent_eval.py`) assesses retrieval quality out of band using a curated 30-prompt set from `data/eval/prompts.yaml`. Prompts are drawn from four categories:

- 10 spatial-grounded prompts (questions about specific suburbs and dimensions)
- 10 no-data prompts (null-scored aspects or unknown suburbs)
- 5 multi-suburb comparison prompts
- 5 out-of-scope prompts

Each prompt is run through `crews.query_crew.run_query`, capturing the answer, the per-suburb evidence trace, and wall-clock latency. An LLM judge evaluates each response against a uniform criterion: *"Does every named-suburb claim in the answer have a supporting trace entry?"* The judge returns a JSON verdict `{retrieval_grounded, reason}`. Results are written to `data/eval/results.csv` with one row per prompt.

**Table 3: Schema of data/eval/results.csv, one row per prompt.**

| Column | Type | Source | Description |
|---|---|---|---|
| id | str | prompts.yaml | kebab-case prompt identifier |
| prompt | str | prompts.yaml | the user question |
| answer | str | agent | the synthesiser's response text |
| retrieval_grounded | bool | judge | every named-suburb claim is supported by a trace entry |
| expect_grounded | bool | prompts.yaml | the labeller's prior verdict for agreement scoring |
| reason | str | judge | the judge's free-text rationale |
| trace_length | int | agent | total evidence_trace entries this turn |
| n_no_data | int | agent | trace entries reporting no_data |
| judge_model | str | CLI | model name used by the judge |
| agent_latency_ms | float | agent | wall-clock time of run_query() in milliseconds |

The uniform judge criterion generalises across categories: an out-of-scope refusal that names no suburbs is trivially grounded, an honest no-data answer with no fabricated claims is grounded, and a hallucinated suburb claim with no supporting trace is ungrounded. The judge's full reasoning is logged per row, enabling manual audit of noisy verdicts without a hand-labelled gold set.

Under the full agentic RAG implementation, trace lengths vary with question type rather than being uniform. The agent terminates earlier on score-lookup questions (typically 1–2 tool calls) and calls `search_posts` on qualitative questions per the prompt-level grounding rule, producing longer traces with resident quote citations on those turns.

Known limitations: LLM-judge verdicts on paraphrased citations are noisy; the 30-prompt set bounds generalisation claims to "directional retrieval-quality evidence on a curated set," not a production benchmark; and the Arc Shift dump is a snapshot, so discourse drift after the dump date is not reflected until the next bulk re-ingest.

### 6.3 User-Facing Application Quality

The dashboard is deployed as a Next.js application on Vercel with the FastAPI backend on Render. The main interface uses a 3-column layout: a 440px chat sidebar, a full-flex Leaflet map panel, and a 320px collapsible EvidenceDrawer. The onboarding flow guides the user through a conversational weight-setting process across five dimensions (safety, transport, lifestyle, affordability, proximity to CBD), persisted in `localStorage` between sessions.

On profile completion, the frontend fetches `/api/civic` with the user's normalised weights and displays the top-5 ranked suburbs as colour-coded polygons on the map (`flyToBounds` centres the

view on the ranked set). Clicking a suburb opens a `ReportModal` with the full suburb report: an 8-dimension sentiment radar chart (`AspectRadar`), a 7-emotion bar chart (`EmotionBars`), Reddit highlight quotes (`RedditQuote`), and a crime breakdown table with trend indicators (`CrimeBreakdown`).

The chat interface streams agent progress via Server-Sent Events (SSE) before delivering the final response. During streaming, progress text updates word-by-word in the `AssistantBubble`; on completion, the full response is passed to `ReactMarkdown` with `remark-gfm` for correct rendering of bold, lists, and blockquotes. The `EvidenceDrawer` shows a pipeline trace: which agents ran, how many chunks were retrieved, latency per specialist, and source freshness badges.

The application was tested through structured walkthroughs covering onboarding, map exploration, single-suburb queries, multi-suburb comparisons, and out-of-scope queries.

### 6.4 Temporal Trends

Reddit posts with `created_utc` timestamps allow limited temporal sentiment analysis where coverage permits. For suburbs with multi-year Reddit history (Newtown, Glebe, Surry Hills, Redfern), the pipeline can surface whether community sentiment on a dimension has shifted over time. BOCSAR crime data covers 2015–2025, enabling year-on-year trend detection at the suburb level; the Crime agent reports these trends as "improving", "worsening", or "insufficient data" based on a comparison of total incidents in the most recent year versus the prior year.

Summarise time-based patterns in Reddit sentiment per suburb where timestamps are available.

---

## 7 Outcomes and Value Added

Sydney Liveability AI delivers a working end-to-end NLP application that addresses a genuine and underserved information gap. For the target user group — international students, migrants, and young professionals choosing a Sydney suburb — the system provides evidence-based answers to questions that no existing tool handles: *"Is Redfern safe?"*, *"Which inner-west suburb is best without a car?"*, *"How do Newtown and Glebe compare for nightlife?"*

The key value delivered is the combination of **resident voice** (20k + Reddit posts indexed and retrieved semantically) with **civic data** (crime, transport, facilities, OSM amenities) in a single conversational interface with cited, auditable evidence. The EvidenceDrawer makes the system's reasoning transparent: users can see exactly which agents ran, which sources were retrieved, and how long each step took — a design choice that builds trust in the answers and distinguishes the system from opaque LLM-based tools.

From a technical standpoint, the project demonstrates a production-grade implementation of agentic RAG at the component level: a ReAct agent with three tools autonomously deciding retrieval strategy on each turn, an evidence trace recorded as a side channel without constraining the agent loop, and a synthesis LLM that composes grounded markdown answers from heterogeneous structured and unstructured inputs. The two-crew CrewAI architecture (offline pipeline crew + real-time query crew) is a scalable pattern that could be extended to additional cities or data sources without restructuring the core system.

The project is hosted and publicly accessible: the frontend on Vercel and the backend on Render, with the GitHub repository available at `https://github.com/Nelkit/sydney-liveability-ai`.

---

## 8 Recommendations and Next Steps

List the highest-impact extensions: coverage of more suburbs, additional data sources, model fine-tuning, and integration with rental price APIs.

The following extensions are recommended in order of estimated impact:

**Automated data refresh.** The most significant limitation of the current system is that all data is a static snapshot. An automated pipeline — scheduled Reddit ingestion via PRAW, BOCSAR data download, and OSM refresh — would keep the system current without manual re-ingestion. The ChromaDB index (~10 hours to rebuild from scratch) could be incrementally updated if chunk IDs remain deterministic.

**Suburb coverage expansion.** Current sentiment coverage is concentrated in inner Sydney. Extending to outer suburbs (Parramatta, Blacktown, Liverpool, Penrith) would require targeted Reddit data collection and potentially augmentation from additional community sources (e.g., local Facebook groups, council consultation submissions from other LGAs).

**Rental price integration.** Integrating current rental price data (e.g., from Domain API or property data providers) would complete the affordability dimension with objective market data, reducing reliance on Reddit-derived affordability sentiment which is subject to recency and selection bias.

**LLM router upgrade.** The current deterministic regex router is fast and cost-free but struggles with ambiguous or creatively phrased questions. Replacing the keyword matching with a lightweight LLM classifier (e.g., a fine-tuned distilled model) would improve routing accuracy on edge cases without significantly increasing latency.

**Model fine-tuning.** The DeBERTa ABSA model was not fine-tuned on Sydney liveability text. Fine-tuning on a small hand-labelled dataset of suburb-specific Reddit posts would likely improve aspect classification confidence, particularly for dimensions with informal vocabulary (e.g., "nightlife", "noise").

**User authentication and personalisation.** Adding a user account system would allow the system to store preferences across sessions and support longer interaction histories. The current `localStorage`-only approach means that profile data is device-specific and not persistent across browsers.

**LLM provider fallback.** The system currently depends on a single default LLM provider. Adding automatic fallback (e.g., OpenRouter -> Anthropic -> OpenAI) would improve production reliability.

---

## 9 Conclusion

Summarise the project contributions, lessons learned, and future potential.

---

## 10 References

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research, 3*, 993–1022.

CrewAI. (2026). *CrewAI* (Version 1.14.4) [Computer software]. https://www.crewai.com/

Du, M., Xu, B., Zhu, C., Wang, S., Wang, P., Wang, X., & Mao, Z. (2026). *A-RAG: Scaling agentic retrieval-augmented generation via hierarchical retrieval interfaces*. arXiv. https://doi.org/10.48550/arXiv.2602.03442

*Hierarchical agentic RAG*. (2026, February 9). Emergent Mind. https://www.emergentmind.com/topics/hierarchical-agentic-rag

Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media, 8*(1), 216–225. https://doi.org/10.1609/icwsm.v8i1.14550

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9475.

Pucek, B. (2026, March 12). LangGraph vs AutoGen vs CrewAI: Choosing the right agent framework for production. *The Thinking Company*.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982–3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv. https://doi.org/10.48550/arXiv.1910.01108

S., E., & Zhang, B. (2024, December 19). Building effective agents. *Anthropic*. https://www.anthropic.com/engineering/building-effective-agents

# A Appendix A: System Architecture Diagram

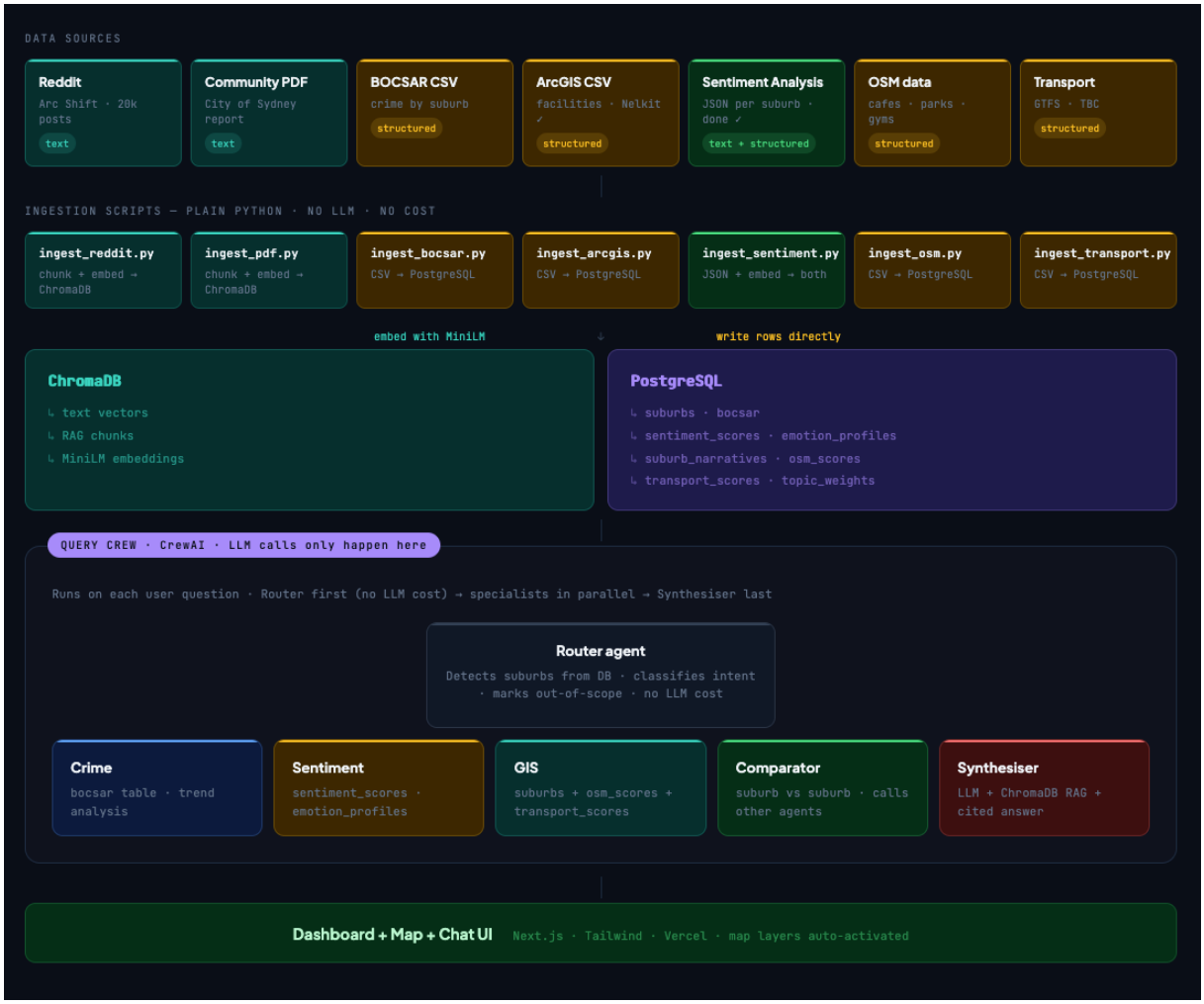


Figure 1: System architecture diagram.

# B Appendix B: Dashboard Screenshots

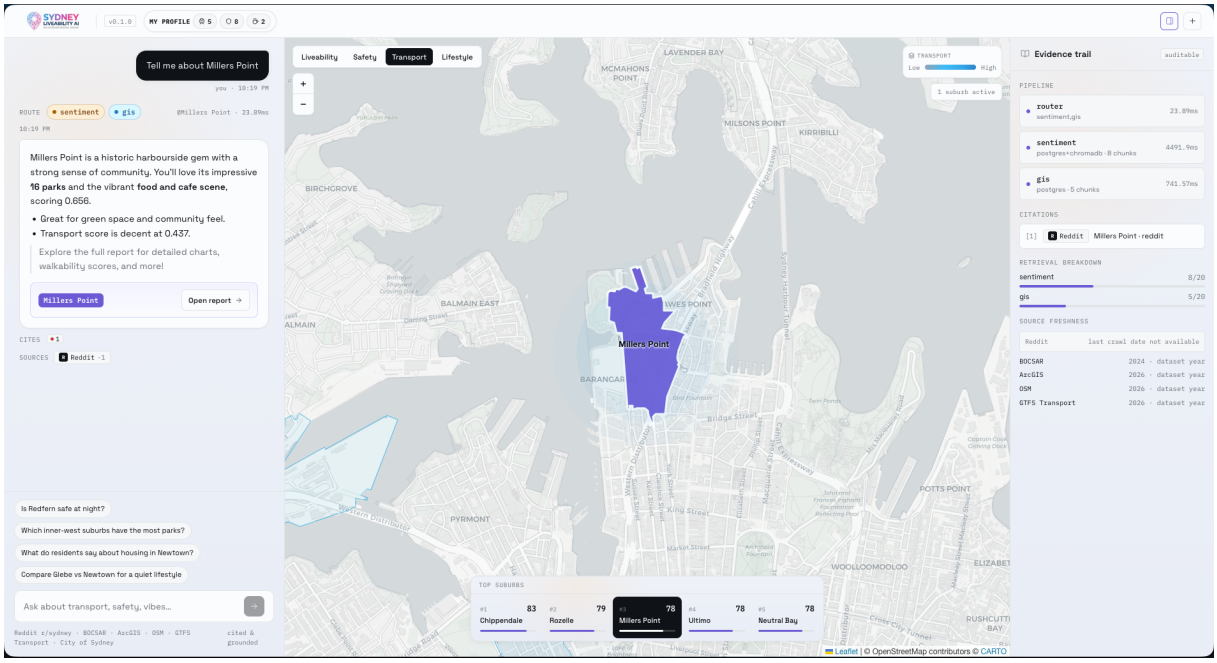


Figure 2: Leaflet map and chat interface.

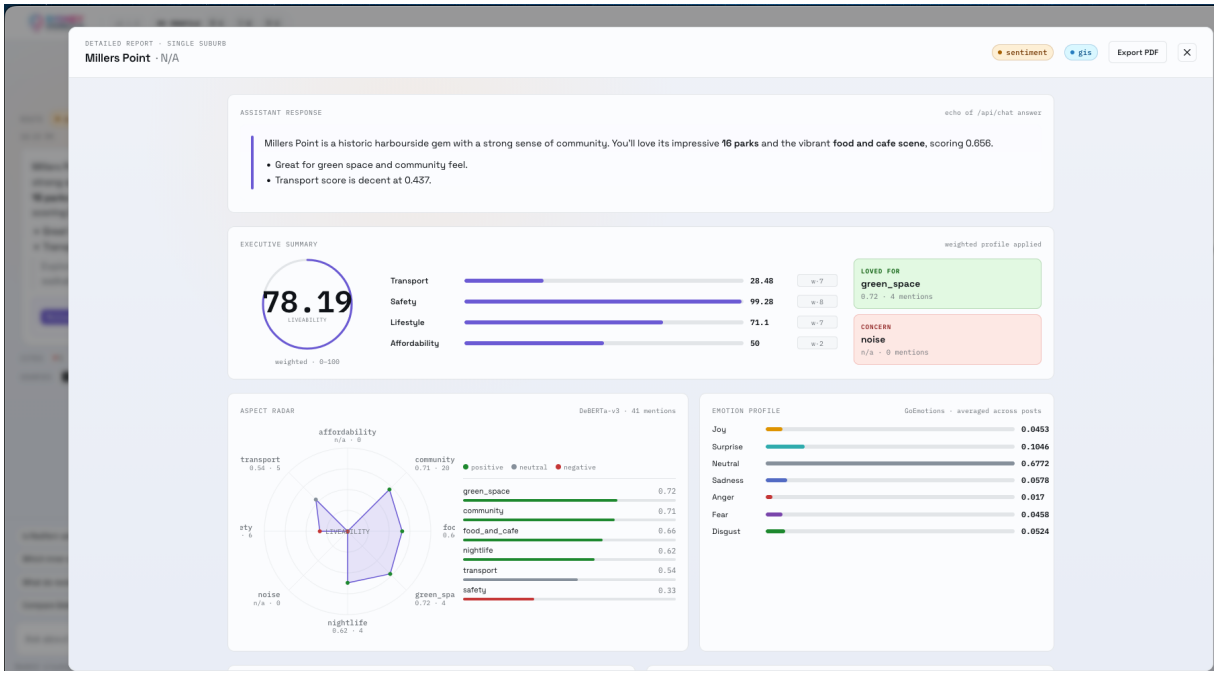


Figure 3: Detailed report with aspect radar and emotion profile.

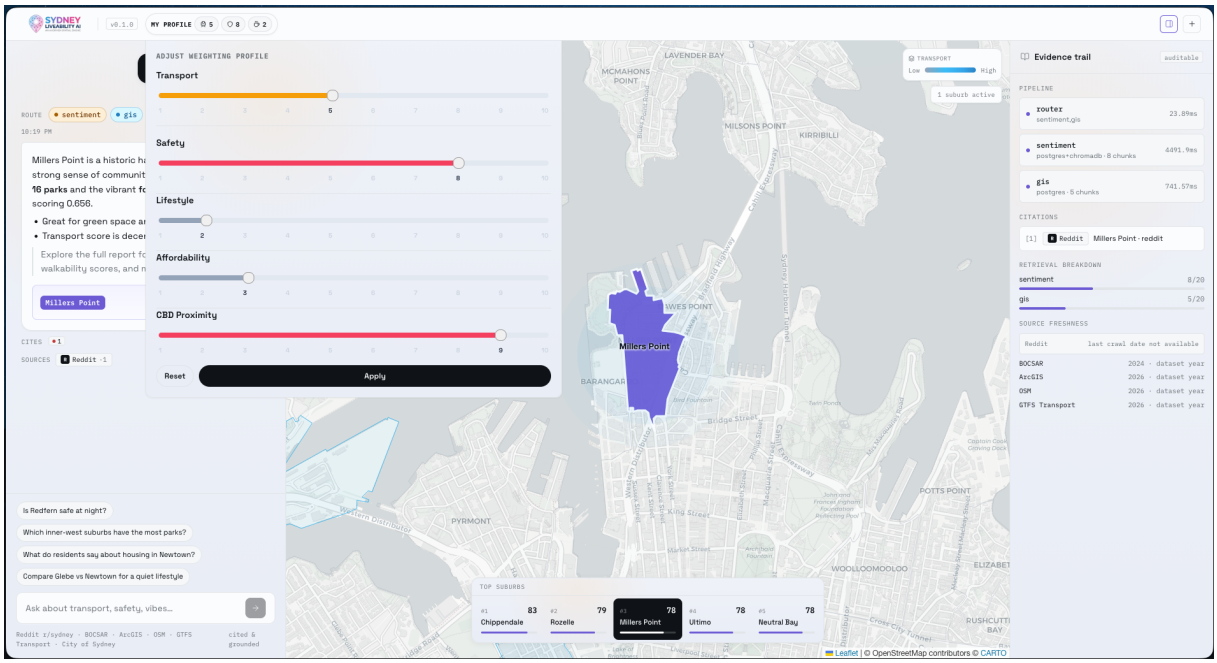


Figure 4: Weight adjustment panel.

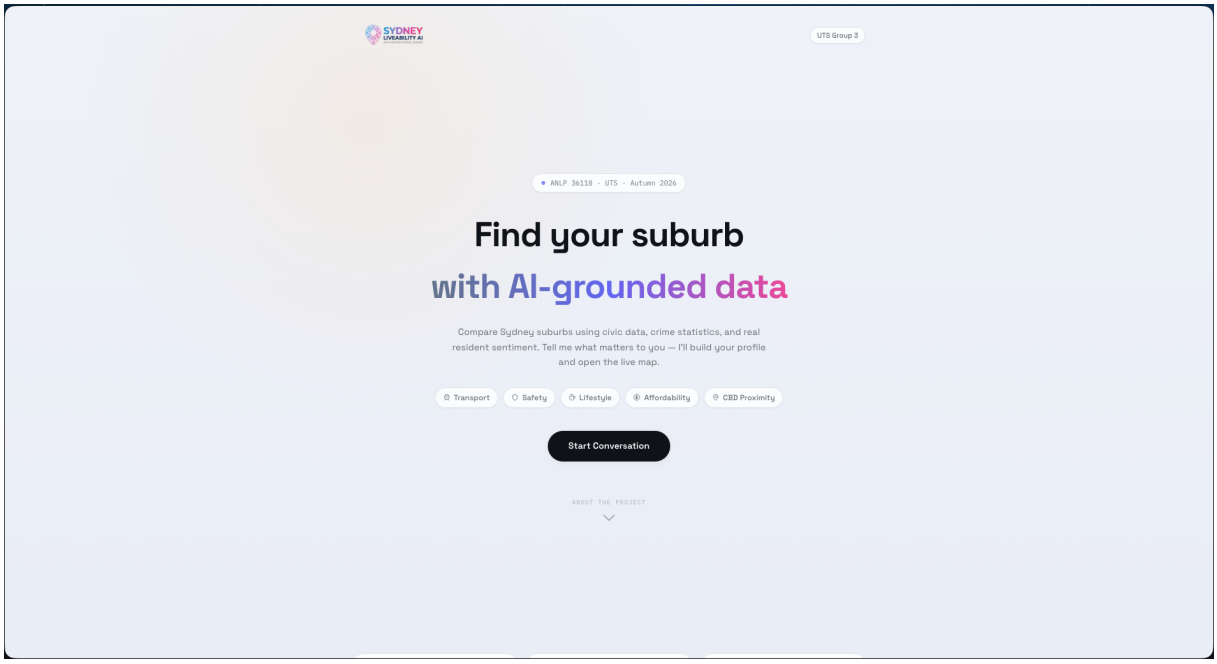


Figure 5: Landing page.

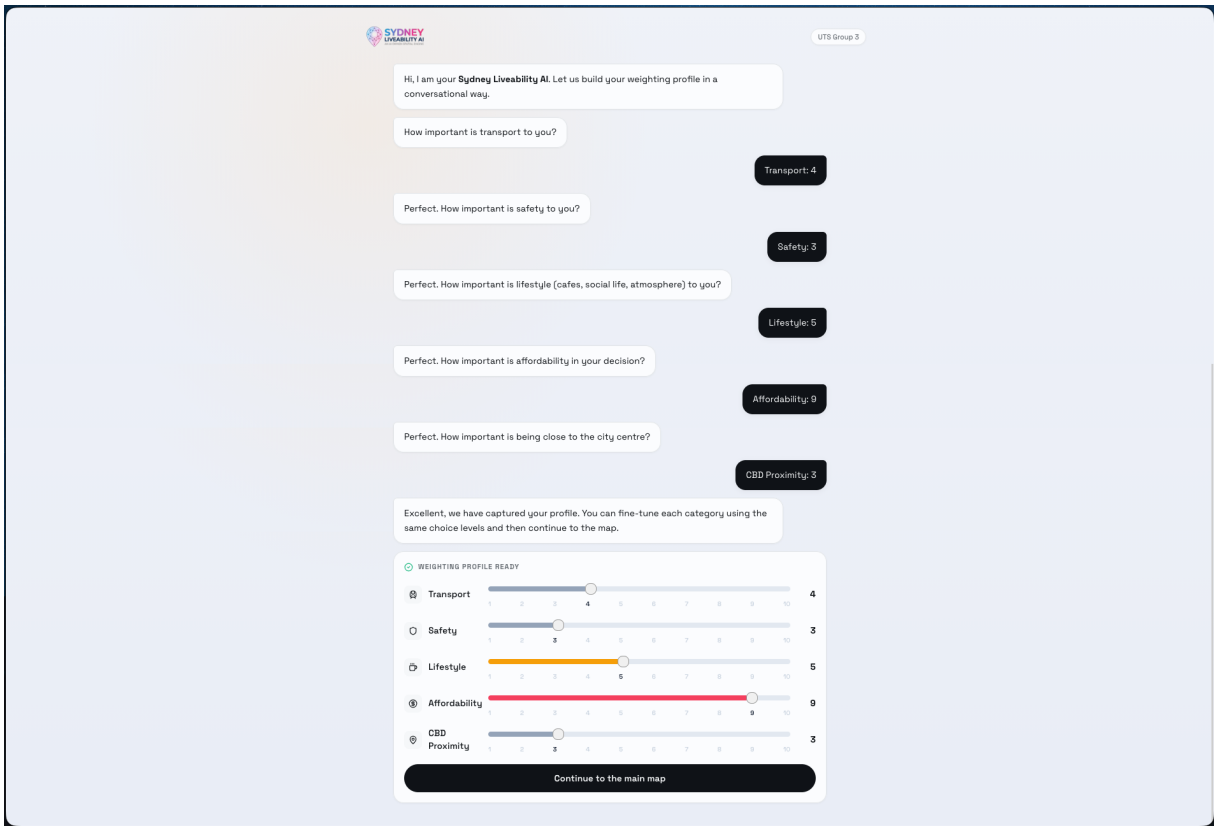


Figure 6: Onboarding page.

---

# C Appendix C: Sample Code

## C.1 ingest_sentiment.py (ingest pipeline)

File: ingest_sentiment.py

```
"""Ingest sentiment-pipeline outputs into ChromaDB.

Reads data/processed/reddit_analyses/{suburb}.json produced by the NLP
pipeline and upserts summaries/quotes into the embedding store.
"""
# ...imports...

def _records_for_narrative(suburb: str, narrative: str) -> list[dict]:
    if not narrative or len(narrative.strip()) < MIN_TEXT_LEN:
        return []
    chunks = chroma.chunk_reddit_text(narrative)
    post_id = _hash_id("narrative", suburb + narrative)
    return [
        {
            "text": chunk,
            "metadata": {
                "suburb": suburb,
                "source": "sentiment_narrative",
                "dimension": "general",
```

```
                "chunk_index": idx,
                "post_id": post_id,
            },
        }
        for idx, chunk in enumerate(chunks)
    ]

def _records_for_quote(suburb: str, quote: dict) -> list[dict]:
    text = (quote.get("text") or "").strip()
    if len(text) < MIN_TEXT_LEN:
        return []
    chunks = chroma.chunk_reddit_text(text)
    url = quote.get("url") or ""
    score = int(quote.get("score") or 0)
    post_id = _post_id_from_url(url) or _hash_id("quote", text)
    return [
        {
            "text": chunk,
            "metadata": {
                "suburb": suburb,
                "source": "sentiment_quote",
                "dimension": "general",
                "chunk_index": idx,
                "post_id": post_id,
                "post_score": score,
                "url": url,
            },
        }
        for idx, chunk in enumerate(chunks)
    ]

def ingest(input_dir=DEFAULT_INPUT_DIR, batch_size=64, suburb_limit=None):
    files = sorted(p for p in input_dir.glob("*.json") if not p.name.startswith("_"))
    buffer = []
    for path in files[:suburb_limit]:
        analysis = json.load(open(path, "r", encoding="utf-8"))
        suburb = analysis.get("suburb") or path.stem.replace("_", " ").title()
        buffer.extend(_records_for_narrative(suburb, analysis.get("narrative", "")))
        for quote in analysis.get("sources") or []:
            buffer.extend(_records_for_quote(suburb, quote))
        if len(buffer) >= batch_size:
            chroma.upsert_chunks(buffer); buffer.clear()
    if buffer:
        chroma.upsert_chunks(buffer)
```

---

## C.2 CrewAI: query crew orchestration

File: query_crew.py

```
def build_query_crew() -> Crew:
    return Crew(
        agents=[
            router_agent,
```

```
            crime_agent,
            sentiment_agent,
            gis_agent,
            comparator_agent,
            synthesiser_agent,
        ],
        process=Process.sequential,
        verbose=True,
    )

def run_query(question: str, weights: dict | None = None) -> dict:
    router_output = run_router({"question": question})
    categories = list(router_output.get("categories", []))
    suburbs = list(router_output.get("suburbs_mentioned", []))

    if "out_of_scope" in categories:
        return run_synthesiser({"question": question, "router": router_output, "outputs": {}})

    specialist_outputs = {}
    for specialist_name in ["crime", "sentiment", "gis"]:
        if specialist_name in categories and suburbs:
            run_func = {"crime": run_crime, "sentiment": run_sentiment, "gis": run_gis}[
    specialist_name]
            specialist_outputs[specialist_name] = {}
            for suburb in suburbs:
                specialist_outputs[specialist_name][suburb] = run_func({"suburb": suburb, "
    question": question})
    # comparator, synthesiser, scoring, and final packaging follow...
```

---

## C.3 CrewAI: sentiment agent (tool wrappers + trace)

File: sentiment.py

```
# Per-call trace accumulator via ContextVar
_TRACE_CTX: contextvars.ContextVar[Optional[list[dict]]] = contextvars.ContextVar("
    sentiment_trace_ctx", default=None)

@tool("get_suburb_aspect")
def _get_suburb_aspect_tool(suburb: str, dimension: str) -> dict:
    result = agent_tools.get_suburb_aspect(suburb=suburb, dimension=dimension)
    _record_call("get_suburb_aspect", {"suburb": suburb, "dimension": dimension}, result,
    elapsed_ms)
    return result

@tool("search_posts")
def _search_posts_tool(suburb: str, query: str, dimension: str = "", k: int = 5) -> dict:
    result = agent_tools.search_posts(suburb=suburb, dimension=dimension or None, query=query,
    k=k)
    _record_call("search_posts", {"suburb": suburb, "dimension": dimension, "query": query, "k"
    : k}, result, elapsed_ms)
    return result

# The agent uses these tools in a ReAct-style loop and exposes an evidence_trace
```

```
# that the crew/orchestrator can summarise for quality instrumentation.
```

---

### C.4 RAG / Retrieval helpers (structured lookup + dense search)

File: tools.py

```
@lru_cache(maxsize=512)
def _load_cached_analysis(suburb: str) -> Optional[dict]:
    # loads SentimentScore, EmotionProfile, SuburbNarrative from Postgres
    # returns structured aspect entries and narrative/sources

def search_posts(suburb: str, dimension: Optional[str], query: str, k: int = 5) -> dict:
    # If the cached analysis marked (suburb,dimension) as null -> short-circuit no_data
    # Otherwise call db.chromadb.query_chunks(query=query, k=k, filters={"suburb": suburb, "
    dimension": dimension})
    hits = _query_chunks(query=query, k=k, filters=filters)
    return {"status": "ok", "suburb": suburb, "results": hits, ...}

def get_suburb_aspect(suburb: str, dimension: str) -> dict:
    # Return structured aspect entry or {"status":"no_data", ...}
```

Also see ingestion/upsert in `chroma.upsert_chunks(...)` invoked by ingest_sentiment.py for embedding-backed RAG.

---

### C.5 FastAPI endpoints (chat + subreddit analysis)

Files: chat.py and reddit_router.py

```
# /api/chat
@router.post("/api/chat")
def chat(payload: ChatRequest) -> dict:
    question = (payload.question or payload.message or "").strip()
    if not question:
        return {"answer": "Please provide a question.", "sources": [], "suburb_scores": [], "
    map_state": None}
    response = run_query(question, payload.weights or {})
    return {
        "answer": response.get("answer"),
        "sources": response.get("sources", []),
        "suburb_scores": response.get("suburb_scores", []),
        "map_state": response.get("map_state"),
        # plus optional debug/quality fields when present
    }

# /api/reddit/{suburb} (on-demand pipeline + cache write)
@router.get("/api/reddit/{suburb}")
def analyse_suburb_endpoint(suburb: str) -> dict:
    normalised = _normalise_suburb(suburb)
    pg_cached = _pg_lookup(normalised)
    if pg_cached is not None:
```

```
        return pg_cached
    # fallback: run pipeline locally and persist
    posts = load_suburb_posts(normalised)
    result = analyse_suburb(normalised, posts)
    response = result.to_dict()
    _pg_write(response)
    _cache_write(_get_supabase(), response)
    return response
```

---

# D Appendix D: Evaluation Tables and Charts

Include suburb-level scores per sentiment dimension, emotion distributions, transport scores, and POI density charts.

---

# E Appendix E: Traditional vs. Modern NLP Comparison Table

**Table 4: Traditional vs. Modern NLP Approach Comparison**

| Approach | Type | Used | Rationale |
|---|---|---|---|
| VADER sentiment | Traditional | No | Lexicon-based, no aspect granularity |
| LDA topic modelling | Traditional | No | Unsupervised, low coherence on Reddit text |
| TF-IDF keyword extraction | Traditional | No | Frequency-based, no semantic understanding |
| DistilRoBERTa aspect sentiment | Modern | Yes | 8-dimension aspect scoring, contextual |
| all-MiniLM-L6-v2 embeddings | Modern | Yes | Semantic similarity, compact and fast |
| RAG with Claude Sonnet | Modern | Yes | Grounded, cited answers from retrieved chunks |
| CrewAI multi-agent pipeline | Modern | Yes | Modular orchestration, scalable per suburb |